

Data Engineer Take Home Project

Project Requirements

You will create a **basic ETL pipeline** that:

1. **Extracts data** from an open API (public REST API)
2. **Transforms the data** to a structured format (JSON or CSV)
3. **Loads the data** into a local file system (simulating a data lake)
4. **Implements basic logging and observability**
5. Push your work to a public GitHub repository and share the link with us (note that we will not accept work that is not turned in through a public GitHub repository).

---

1. Data Extraction (Ingestion)

- Choose an **open API**. Any one you like, or the following:
  - https://jsonplaceholder.typicode.com/
  - https://openweathermap.org/api
  - https://randomuser.me/
  - https://alexwohlbruck.github.io/cat-facts/
- Write a **Go package** to fetch data from the API every 30 seconds.
- Store the data into a Postgres Database

2. Data Extraction

- **Write additional Go function to export processed data into local storage.**
  - You can store raw data **locally in a JSON** (e.g., `data/raw_data.json`)

3. Data Transformation

- Process the extracted data to **remove unnecessary fields** and normalize the structure in any way you want. Be ready to explain your choice of structuring - think of more complex structures in the future.
- Convert timestamps into a **consistent format (e.g., UTC, ISO 8601)**.
- Save the transformed data in a **separate file** (e.g., `data/processed_data.json`).

4. Data Storage (Local Simulation of a Data Lake)

- Raw data should be stored under `data/raw/` and transformed data under `data/processed/`.
- Ensure that the data is **appended instead of overwriting**.

5. Observability & Logging

- Implement **logging** for:
  - API request success/failure
  - Transformation errors
  - Data saved successfully
- Save logs into a `logs/etl.log` file using Go's logging package.
- Go package exposes endpoints for health and metrics (Prometheus style) monitoring.
  - Be ready to explain metrics included in monitoring.

5. Containerization

- Create a Dockefile and make sure your go package can run in a docker container
- When launching the docker container make sure you add steps on how to mount the above required volumes to simulate persistency.

6. README.md

- Include a detailed README.md
- Include a section on how you would productionize
  - You may include what cloud related services you would use for which components

- How you would account for scalability and reliability

TIP: The more detailed the README.md the better